



Git Journey

Weather App — Práctica de Git con Java

Java | Git

¿Qué hemos hecho?

La App

- Consultar el clima de ciudades
- 10 ciudades predefinidas
- Datos aleatorios para el resto
- Historial de consultas
- Limpiar historial
- Todo desde consola

Las clases Java

- WeatherApp.java — main()
- WeatherData.java — modelo
- WeatherService.java — lógica
- WeatherHistory.java — historial

Hecho con ayuda de Claude AI
Código limpio y comentado

Paso 1 — git init

Primero creamos la carpeta del proyecto y lo inicializamos con Git:

```
$ mkdir weather-app && cd weather-app
$ git init
Initialized empty Git repository in ~/weather-app/.git/

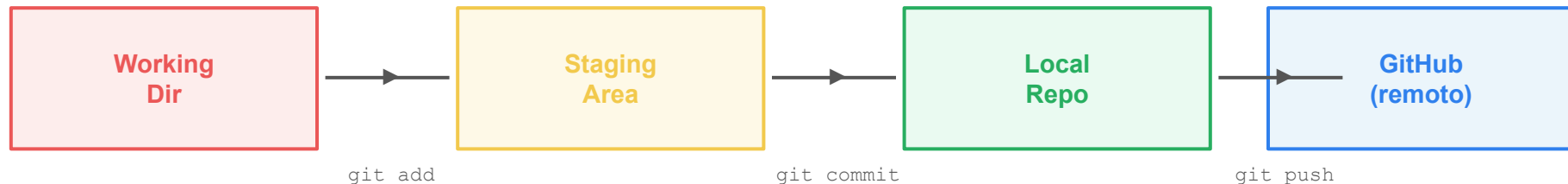
$ git status
On branch main — No commits yet
Untracked files: src/  README.md
```

¿Qué pasa aquí?

git init crea la carpeta oculta .git/ que es donde Git guarda todo el historial.

git status nos dice qué archivos tiene Git en cuenta (rojo = sin rastrear todavía).

Paso 2 — git add y git commit



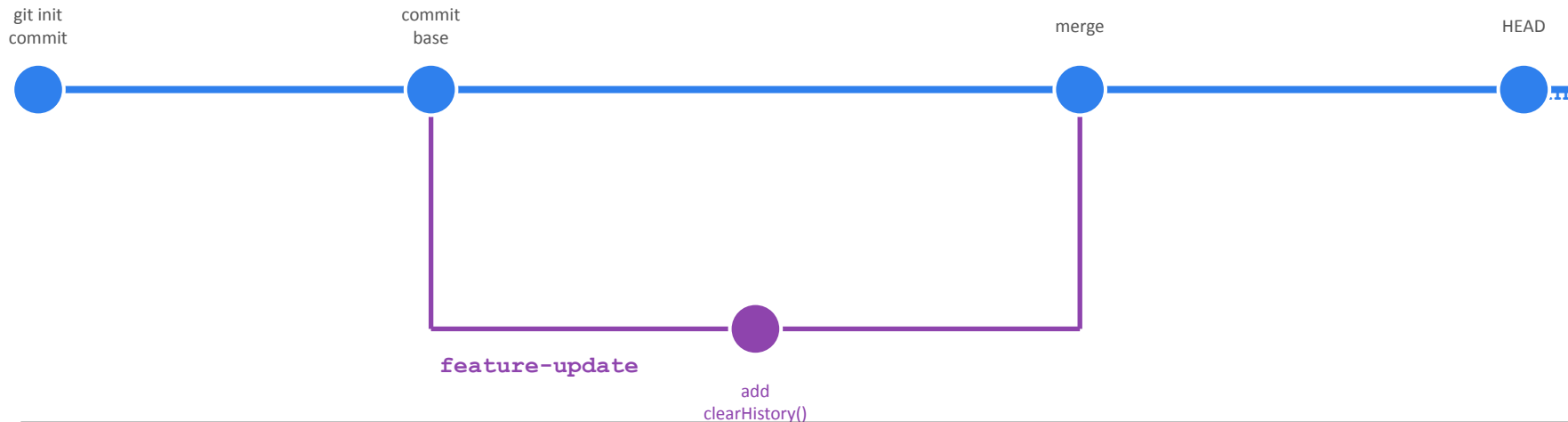
```
$ git add .
$ git status
Changes to be committed:
  new file:   WeatherApp.java
  new file:   WeatherData.java
  new file:   WeatherService.java

$ git commit -m "Initial commit: WeatherApp base"
[main a3f1c8b] Initial commit - 5 files changed, 280 insertions(+)
```

El punto (.) en git add . significa «todos los archivos». Los archivos pasan de rojo a verde en git status.

Paso 3 — Ramas (Branching)

Creamos la rama feature-update para añadir la función clearHistory():



```
$ git checkout -b feature-update      # crear y cambiar
$ git add .    &&  git commit -m "Add clearHistory feature"
$ git checkout main    &&  git merge feature-update    # fusionar
```

Paso 4 — Subir a GitHub

```
# Vincular con GitHub
$ git remote add origin
  https://github.com/USUARIO/weather-app

# Verificar
$ git remote -v
origin  https://github.com/... (fetch)
origin  https://github.com/... (push)

# Subir todo
$ git push -u origin main
* [new branch] main -> main
```

1

Crear repo en GitHub (público, sin README)

2

git remote add origin <url>

3

git push -u origin main

4

Settings → Pages → main → / (root)

GitHub Pages

Una vez activado en Settings, el sitio queda disponible en: https://TU_USUARIO.github.io/weather-app
Ahí se sirve el archivo index.html del repo como landing page del proyecto.

El historial — git log

Así queda el historial de commits al final del proyecto:

```
$ git log --oneline --graph --decorate

*    e9d2f1a (HEAD -> main, origin/main) Merge branch 'feature-update'
| \
| * 7c4a3b9 (feature-update) Add clearHistory feature
| /
*    b2e1d8c Add WeatherHistory class and menu option 3
*    9f3a71e Add WeatherService with city database
*    5c8b2d1 Add WeatherData model class
*    a3f1c8b Initial commit: WeatherApp base + README
```

5 commits

2 ramas

1 merge

0 conflictos

Resumen

Todo lo que hemos hecho en este proyecto:

- git init — repositorio local creado

- git add / commit — código versionado

- git branch / merge — rama feature-update mergeada

- git push — historial subido a GitHub

- GitHub Pages — landing page publicada

- README.md — documentación lista